07 — Architecture Decision Records

Task 7 — Sprint 2 — Foundation & Planning

These ADRs are **binding**. Implementation must follow them; deviation requires a new ADR that supersedes the old one. Format: Context → Decision → Alternatives → Consequences.

ADR	Decision	Status
ADR-001	LangGraph state machine for orchestration	Accepted
ADR-002	pgvector inside the application Postgres as the vector store	Accepted
ADR-003	PostgreSQL 16 as the relational store, accessed read-only	Accepted
ADR-004	OpenAI-compatible LLM behind a provider-agnostic client, tiered models	Accepted
ADR-005	Named parameterized SQL templates instead of text-to-SQL	Accepted
ADR-006	Short-term session memory in Postgres; no long-term vector memory	Accepted
ADR-007	Deterministic guardrail and escalation rule engine; LLM only for summarization	Accepted
ADR-008	Typed citation objects and tool-less synthesis	Accepted
ADR-009	structlog + OpenTelemetry + Prometheus for observability	Accepted
ADR-010	Two-tier latency budget with category-conditional depth	Accepted
ADR-011	React frontend deferred to Sprint 5+ behind frozen API contract	Accepted
ADR-012	In-repo evaluation harness rather than a third-party eval platform	Accepted
ADR-013	Hybrid retrieval (dense + sparse + rerank) over pure dense	Accepted
ADR-014	RBAC enforced at the SQL executor boundary, not in prompts	Accepted
ADR-015	Twelve-factor / cloud-portable runtime (stateless app, env config)	Accepted
ADR-016	Secrets via env injection; secret manager in cloud; stub blocked in prod	Accepted
ADR-017	One container image; CI gates; deploy manifests separate from app code	Accepted
ADR-018	Corpus behind storage Protocol; local FS now, object storage later	Accepted
ADR-019	Self-serve signup + login; admin assigns role/portfolios	Accepted
ADR-020	Separate auth DB from finance/vector DB	Accepted
ADR-021	Stock screening module (Screener-like) with whitelist SQL	Accepted
ADR-022	Capstone and Production are **separate plans** (not one dual-mode app)	Accepted
ADR-023	React app: login/signup first; protected routes for all features	Accepted
ADR-024	Cursor planning/implementation: Opus 4.5 or Codex High	Accepted

ADR-001 Use LangGraph as the multi-agent orchestration framework

Status: Accepted **Date:** Sprint 2 **Deciders:** Architect, Backend lead

Context

The system needs 7 collaborating agents (Router, Retrieval, Market Data, Portfolio Evaluation, Risk, Synthesis, Escalation) with parallel fan-out for hybrid queries, a hard terminal escalation state, and measurable Query Routing Accuracy (>=95%). It must be testable node-by-node and replayable from a trace.

Decision

Use **LangGraph** with an explicit `GraphState` and conditional edges. Use **LangChain only for tool and LLM adapters** not for its higher-level agent abstractions. Paths are declared, not discovered: one subgraph per query category.

Alternatives considered

Option	Why rejected
CrewAI / AutoGen conversational crews	Agent-to-agent chat is nondeterministic and hard to bound; routing accuracy and per-node latency attribution become guesswork; escalation cannot be a guaranteed terminal state
Plain LangChain `AgentExecutor` (ReAct loop)	Unbounded tool loops break the 2s budget and make cost unpredictable; no first-class parallel fan-out
Hand-rolled `asyncio` orchestrator	Would have to reimplement state merging, conditional edges, checkpointing, and streaming cost not justified
Single mega-prompt with function calling	Cannot hit 95% SQL correctness or produce per-agent traces; safety checks become prompt instructions

Consequences

Positive: deterministic, unit-testable nodes; native parallel fan-out for C3/C4; escalation as a real terminal node; per-node spans give exact latency attribution; state is serializable so runs are replayable. **Negative:** more boilerplate than a conversational crew; LangGraph API churn is a maintenance risk (pin the version); new paths require editing the graph rather than "just prompting". **Mitigation:** keep node functions thin and framework-agnostic (pure `state ? state` functions) so a future orchestrator swap touches only `graph/build.py`.

ADR-002 Use pgvector inside the application Postgres as the vector store

Status: Accepted **Deciders:** Architect, DB lead

Context

The RAG corpus is small and static: 7 PDFs, an estimated 400 900 chunks. The system already requires PostgreSQL for the 5 business tables. Citations must join chunk metadata (`doc`, `section`, `page`) and retrieval must be filterable by document category.

Decision

Use **PostgreSQL 16 with the `vector` extension** (image `pgvector/pgvector:pg16`), table `policy_chunks`, HNSW index with cosine distance, plus a GIN full-text index on `chunk_text` for the sparse half of hybrid retrieval (ADR-013).

Alternatives considered

Option	Why rejected
Chroma (local persistent)	A second store to run, back up, and keep in sync; metadata cannot be SQL-joined; two connection models for one small corpus
Qdrant / Weaviate	Operationally heavier than a 900-chunk corpus warrants; another container, another client
FAISS in-process	No metadata filtering, no persistence story, no concurrent writes, rebuilt on every boot
Dedicated managed vector DB (Pinecone, etc.)	Extra hop, cost, and dual backup story for a small corpus; conflicts with single-DB ops
Cloud-only vector product with no local parity	Breaks local/CI parity (ADR-015)

Consequences

Positive: finance knowledge in one DB (tables + vectors joinable); ACID re-ingest; hybrid retrieval in a single query; HNSW fine at this scale. **User/auth data is NOT here** — see ADR-020. **cloud path:** managed Postgres 16 + `vector` for `finance_risk_db` only. **Negative:** finance/vector Postgres needs the extension installed (a plain `postgres:16` image will not do); at very large corpora pgvector would need tuning. **Fallback clause:** if the target environment cannot provide the `vector` extension, fall back to Chroma behind the existing `rag/store.py` interface. This is the *only* sanctioned substitution and must be recorded in the eval report header (retrieval metrics are not comparable across stores).

ADR-003 PostgreSQL 16 as the relational store, accessed through a SELECT-only role

Status: Accepted **Deciders:** Architect, DB lead

Context

The dataset spec defines 5 tables with `SERIAL` primary keys, `NUMERIC`, `BOOLEAN`, foreign keys, and a `CHECK (risk_score BETWEEN 0 AND 100)` constraint. An LLM-driven system will be issuing queries against this data, and Unauthorized Data Access must be 0 with no possibility of mutation.

Decision

Use **PostgreSQL 16**, exactly the DDL from the dataset specification. The application creates two roles: `finrisk_app` (owner, used for migrations and ingestion) and `finrisk_readonly` (SELECT-only, used by the SQL executor on the answer path). Add indexes on the columns the golden queries filter by: `portfolio_holdings(portfolio_id)`, `portfolio_holdings(sector)`, `asset_risk_logs(asset_name)`, `asset_risk_logs(risk_score)`, `asset_risk_logs(escalation_flag)`, `market_indicators(indicator_name, region)`, `investment_decisions(portfolio_id)`, `investment_decisions(approval_status)`.

Alternatives considered

Option	Why rejected
SQLite	No `SERIAL`/`NUMERIC` semantics, no roles, no pgvector, no realistic concurrency would diverge from the given DDL
MySQL	Would require rewriting the provided DDL; no pgvector
DuckDB	Analytical, embedded; no role-based grants, no realistic multi-user story

Consequences

Positive: the dataset DDL runs unmodified; database-enforced read-only on the answer path is a structural guarantee, not a code convention; FK from `investment_decisions.risk_assessment_reference` to `asset_risk_logs.risk_id` supports decision-to-risk traceability. **Negative:** two roles to manage; a developer running the app as `finrisk_app` by accident would lose the read-only guarantee asserted by a startup check and an integration test that expects an INSERT to fail.

ADR-004 OpenAI-compatible LLM behind a provider-agnostic client, with tiered models

Status: Accepted **Deciders:** Architect, AI Eng lead

Context

Different nodes have different needs. Routing needs speed and consistency on a 5-class problem. Risk reasoning and synthesis need capability. The eval judge must not be the same configuration as the model being judged. Provider availability may change mid-project.

Decision

All model access goes through `app/core/llm.py`, a thin OpenAI-compatible client configured by `LLM_BASE_URL` / `LLM_API_KEY`. Three configured tiers: - `LLM_MODEL_ROUTER` small/fast, used by Router and entity extraction - `LLM_MODEL_AGENT` capable, used by Retrieval query expansion, Risk, Portfolio Eval, Synthesis - `LLM_MODEL_JUDGE` used only by the eval harness

Temperature 0 everywhere on the answer path. Embeddings: `text-embedding-3-small` (1536 dims). The client owns retries (2, exponential backoff), timeouts, and token/cost accounting.

Alternatives considered

Option	Why rejected
One model for everything	Either too slow/expensive for routing or too weak for risk reasoning
Direct SDK calls scattered through nodes	No central cost accounting, no swap point, no uniform retry policy
Local model (Ollama) as primary	Insufficient instruction-following for a 16-SLO safety-critical evaluation within the timeline; kept as an offline-dev option only
Anthropic/Gemini native SDKs	Fine models, but committing to a native SDK loses the single-swap-point property; use them through an OpenAI-compatible gateway if desired

Consequences

Positive: provider swap is one env change; per-node cost visibility; judge independence is configurable; deterministic (temp 0) runs make eval comparisons meaningful. **Negative:** provider-specific features (native caching, structured-output extensions) are not used; metrics are model-dependent, so every eval report must record the model triple in its header.

ADR-005 Named parameterized SQL templates instead of LLM-generated SQL

Status: Accepted **Deciders:** Architect, DB lead **Drives:** SQL Correctness >=95%, RBAC = 0 violations

Context

20 of the 50 golden queries (C2 plus the SQL half of C3/C4) depend on structured results. Free-form text-to-SQL on a 5-table schema typically lands in the 70 85% range on non-trivial joins and aggregations structurally short of the 95% target and it makes RBAC scoping and injection defence probabilistic.

Decision

Maintain a **template registry** (`app/sqlq/templates.py`): each entry is `{query_id, description, tables[], sql (with named placeholders), slots{name: type+validator}, rbac_scoped: bool}`. The LLM's only structured-data job is to **select a `query_id` and fill validated slots**. The executor validates every slot against its declared type/enum/range, injects the RBAC portfolio-id filter, applies `MAX_SQL_ROWS`, executes as `finrisk_readonly`, and writes an audit record. If no template fits, the system says so and (for C3/C4) escalates rather than improvising SQL.

Initial registry (~18 templates) covers: portfolios by risk category / manager / benchmark; holdings by portfolio / sector / allocation threshold; sector concentration aggregation; risk logs by score threshold / level / asset / escalation flag / date range; market indicators by name+region / latest by region / trend direction; decisions by portfolio / approval status / type / joined to risk reference.

Alternatives considered

Option	Why rejected
Full text-to-SQL with an LLM	Cannot reach 95% correctness reliably; unbounded query shapes make RBAC and cost unpredictable; injection surface

Option	Why rejected
Text-to-SQL + validator/repair loop	Adds latency and still leaves a long tail of semantically-wrong-but-valid SQL
ORM query builder driven by the LLM	Same unbounded-shape problem wearing a nicer hat
Pre-computed materialized views only	Too rigid for the parameter ranges the golden set requires

Consequences

Positive: correctness becomes a fixed, testable set each template gets a unit test with expected rows; injection surface is effectively closed; RBAC is enforced in one place; query plans are known and indexable; SQL Correctness becomes achievable at 95%+. **Negative:** genuinely novel questions outside the registry cannot be answered accepted, and handled by an explicit "no template covers this" response; the registry must grow as golden queries demand. **Note:** the registry is the *coverage contract*. Every C2/C3/C4 golden query must map to at least one `query_id`, asserted by a test.

ADR-006 Short-term session memory in Postgres; no long-term vector memory

Status: Accepted **Deciders:** Architect, Backend lead

Context

Analysts ask follow-ups ("and for the Balanced Income Fund?"). But per assumption A9, single-turn queries dominate and no SLO depends on memory. Long-term memory would introduce a stale-context risk: a cached risk score from last week presented as current is a safety problem in this domain.

Decision

Two-part memory, both bounded: 1. **Session memory** (lives in **`finance_auth_db`**, ADR-020 — not in the vector DB) tables `sessions` and `turns` in Postgres, holding the last **5 turns** per session: query, category, resolved entities, cited sources, escalation status. TTL 60 minutes. 2. **Entity carry-over** resolved entities (portfolio ids, asset names, sector, region) are carried into the next turn's Router context so pronouns and ellipsis resolve.

No long-term user-preference memory. No vector memory of past conversations. Retrieved evidence and SQL results are never reused across turns every turn re-fetches. Prompt caching of *static* policy chunks is permitted as a latency optimization; caching of *answers* or *risk conclusions* is not.

Alternatives considered

Option	Why rejected
Full conversation history in the prompt	Token cost and dilution; injection risk from earlier turns; breaks the 2s budget on long sessions
Vector store of past Q&A pairs	Reuse of a stale risk conclusion is a safety failure in a financial risk system

Option	Why rejected
Summarization memory (rolling LLM summary)	Extra LLM call per turn against a 2s budget, for a benefit no SLO requires
Stateless, no memory at all	Rejected only because follow-up ergonomics are cheap to provide with a 5-turn window

Consequences

Positive: stale-data risk eliminated for evidence; bounded token cost; app stays stateless (memory in Postgres) so replicas scale freely; sessions are auditable. **Negative:** long multi-turn investigations lose early context after 5 turns; memory is not personalized. Both accepted for the capstone scope.

ADR-007 Deterministic guardrail and escalation rule engine; LLM used only for summarization

Status: Accepted **Deciders:** Architect, Risk/Compliance reviewer **Drives:** Escalation Recall = 1.0, Guardrail Effectiveness = 1.0, CMR < 3%

Context

Three SLOs demand absolute values. A probabilistic gate cannot deliver 1.0: an LLM asked "should this escalate?" will, at some rate, say no. The escalation conditions in this domain are also genuinely checkable in code `escalation_flag`, `risk_score`, `approval_status`, allocation vs. a cited limit.

Decision

Implement `app/guardrails/escalation_rules.py` as a **pure deterministic rule engine** evaluating triggers **E1 E8** (defined in `10_Detailed_Solution_Design.md` 4) after every evidence-producing node. Input guardrails are two-stage: a deterministic pattern layer (override / ignore / bypass / waive / skip assessment / despite escalation_flag / without approval / regardless of policy, plus paraphrase patterns) followed by an LLM intent classifier that can only **add** a trigger, never remove one. The LLM's role in escalation is limited to **writing the human-readable handoff summary** after the decision to escalate has already been made in code.

Alternatives considered

Option	Why rejected
LLM-judged escalation	Cannot guarantee 1.0 recall; failure mode is a silent miss
Third-party guardrails library (NeMo, Guardrails AI)	New dependency for rules that are ~150 lines of domain-specific `if`s; the domain rules (escalation_flag, Basel thresholds) would still be custom
Post-hoc escalation (answer then warn)	The unsafe answer has already been produced and shown
Classifier model fine-tuned for escalation	Out of scope (no fine-tuning) and still probabilistic

Consequences

Positive: 1.0 recall is structurally achievable and provable by unit test; rules are reviewable by a compliance reader who does not read Python well; each trigger has a stable id for logging and metrics; a missed escalation is always a code bug with a reproducible test, never "the model's mood". **Negative:** rules are brittle to novel phrasings of bypass intent mitigated by the LLM classifier as an additive second layer and by permanently growing the attack suite; deterministic rules will over-escalate in some borderline cases. **We explicitly accept over-escalation (lower precision) in exchange for perfect recall.**

ADR-008 Typed citation objects and a tool-less Synthesis agent

Status: Accepted **Deciders:** Architect, AI Eng lead **Drives:** Source Attribution Rate = 1.0, Faithfulness >=80%

Context

"Include sources" as a prompt instruction produces prose citations that cannot be validated automatically, and an agent with retrieval tools at synthesis time can silently introduce claims whose provenance was never recorded.

Decision

1. Citations are **typed objects**, not prose: `DocCitation{doc_id, doc_title, section, page, chunk_id, score}` and `SqlCitation{query_id, tables[], params, row_count, executed_at}`. Rendered as `[DOC: 02_Risk_Management_Exposure_Policy.pdf  Exposure Limits  p.12]` and `[SQL: asset_risk_logs query_id=risk_logs.by_score_threshold  rows=7]`.
2. The **Synthesis agent has no tools**. It receives the evidence bundle and composes from it only.
3. An **output validator** runs after synthesis: at least one resolvable citation; every sentence containing a number, percentage, threshold, or asset name carries a citation; every cited `chunk_id`/`query_id` exists in the bundle. Failure ? one repair retry ? refusal template.

Alternatives considered

Option	Why rejected
Prompt-instructed prose citations	Unverifiable; hallucinated page numbers are common
Post-hoc attribution (attribute after generating)	Attribution becomes a guess about text already written
Inline footnote markers only	Not machine-resolvable back to a chunk
Synthesis with retrieval tools for gap-filling	Reintroduces unprovenanced claims; makes Faithfulness unmeasurable

Consequences

Positive: Source Attribution Rate is mechanically measurable and enforceable at 1.0; Faithfulness is well-defined (claims vs. bundle); the UI can render clickable citations in Sprint 9 with no extra backend work; a wrong citation is detectable rather than plausible. **Negative:** answers are more clipped and less fluent; if

retrieval misses, synthesis cannot rescue it it must refuse (correct behaviour, but it shifts pressure onto retrieval quality); the validator can false-positive on incidental numbers (dates, counts of rows) mitigated with a numeric-context whitelist.

ADR-009 structlog + OpenTelemetry + Prometheus for observability

Status: Accepted **Deciders:** Backend lead

Context

Tasks 20, 22, 42 require operational logging, tracing, cost tracking, and observability validation. A financial governance context requires answering "why did the system say that, using what evidence, at what cost?" weeks later.

Decision

- **Logs:** `structlog` JSON. One request line + one line per graph node, each with `trace_id`, `session_id`, `role`, `category`, `path`, `agents_used`, `escalation{required,trigger}`, `citation_count`, `latency_ms`, `token_usage`, `cost_usd`. Never log secrets or full chunk text (log `chunk_id`s).
- **Traces:** OpenTelemetry span per graph node, attributes for template `query_id`, retrieved `chunk_id`s, model, tokens. `trace_id` returned in every API response.
- **Metrics:** `prometheus-client` at `/metrics`: request counter by `category`/`outcome`, latency histogram by `category`, escalation counter by `trigger`, guardrail-block counter by `attack_type`, token/cost counters by `agent`.
- **Audit:** an `audit_log` row per request persisting the decision record (query hash, category, citations, escalation, role, scope) for Compliance Officer review.

Alternatives considered

Option	Why rejected
LangSmith / Langfuse	Excellent LLM-native tracing, but an external dependency and data egress of financial queries; OTel keeps traces local and vendor-neutral
Plain `logging` + print debugging	Cannot satisfy observability validation or per-node latency attribution
APM SaaS agent	Cost and egress for a capstone

Consequences

Positive: vendor-neutral, local-first, exportable to any OTLP sink later; per-node latency attribution makes latency work data-driven; cost per category is visible from Sprint 4. **Negative:** LLM-specific niceties (prompt diffing, playgrounds) must be hand-rolled; span discipline is a manual convention enforced by a decorator on node functions.

ADR-010 Two-tier latency budget with category-conditional depth

Status: Accepted **Deciders:** Architect, Backend lead, Product reviewer

Context

The tracker sets Latency <=2s. A C4 High-Risk query legitimately requires 3 parallel evidence agents, a risk reconciliation call, and a synthesis call. At realistic per-call latencies that is 4 6s. Meeting 2s for C4 would require removing agents, i.e. sacrificing the reasoning quality the project exists to demonstrate. Silently missing the SLO, or gaming it by shrinking the chain, are both worse than being explicit.

Decision

Declare the SLO precisely: - **Latency SLO (<=2s, p95) applies to C1 (RAG) and C2 (SQL)** the single-hop paths, which are 25 of 50 golden queries and the majority of expected real traffic. - **C3 Hybrid:** documented secondary budget **<=4s p95**. - **C4 High-Risk multi-agent:** documented secondary budget **<=6s p95**. - **C5 Escalation:** <=1.5s p95 (deterministic path, minimal model use). Latency is measured server-side, request-in to response-out, warm index, excluding client network. Techniques to hold the budgets: parallel fan-out, small model for routing, policy-chunk caching, `RETRIEVAL_FETCH_K` tuning, streaming first-token for perceived latency (Sprint 9). **Never** reduce guardrails, citations, or escalation checks to meet a budget.

Alternatives considered

Option	Why rejected
2s for all categories	Achievable only by collapsing multi-agent reasoning defeats the project's purpose
Ignore the mismatch and report a blended p95	Dishonest; hides C4 behaviour behind C1/C2 volume
Async job + polling for C4	Better UX honesty, but adds an endpoint, a queue, and frontend complexity the timeline cannot absorb; revisit post-Sprint-10

Consequences

Positive: honest, measurable, per-category reporting; optimization effort targets the paths users hit most; the architecture keeps its reasoning depth. **Negative:** the headline SLO now needs a footnote must be stated in every eval report and in `09_Detailed_Use_Case_and_SLO.md` 4; a reviewer skimming for "Latency <=2s: PASS" must see the scope qualifier. Every eval report prints the per-category table, not a single number.

ADR-011 Frontend deferred to Sprint 5+ behind a frozen API contract

Status: Accepted **Deciders:** Architect, PM

Context

The tracker places the frontend skeleton at Sprint 5 (Task 30), integration at Sprint 7, and full UX at Sprint 9. Sprints 4 6 are where all SLO risk lives. The project window is roughly four weeks.

Decision

Use **React (Vite + TypeScript + React Router)**. No UI before Sprint 5. Task 30 delivers auth pages (login/signup) as the **only public entry** plus a frozen API contract (`14`, `19`). Full feature UI in Sprints 7–9. See ADR-023 for route guards.

Alternatives considered

Option	Why rejected
Build the UI in parallel from Sprint 4	Contract churn plus split attention during the highest-risk sprints
Ship a Streamlit demo instead of a real frontend	Fine for a demo, but Tasks 43/44/50/51 require real auth, RBAC, and citation UX
Contract-last (design the API after the UI)	Would invalidate the eval harness written against the API

Consequences

Positive: backend and eval velocity in Sprints 4 6; the UI is built against a stable, documented contract; the envelope is UI-complete from Sprint 3. **Negative:** no visual demo until Sprint 5 stakeholder demos in Sprint 4 use `/docs` and eval reports; any envelope oversight in Sprint 3 costs an additive change later.

ADR-012 In-repo evaluation harness rather than a third-party eval platform

Status: Accepted **Deciders:** AI Eng lead, Architect

Context

16 SLOs must be measured every sprint, four of them as hard CI gates. Several are domain-specific (Risk Classification Accuracy against `asset_risk_logs` bands, Escalation Recall against E1 E8, SQL Correctness by result-set comparison) and no off-the-shelf platform computes them.

Decision

Build `eval/` in-repo: `python -m eval.run --suite {golden|attack|negative|regression|all}`. Implement RAGAS-*style* metrics (faithfulness, answer relevance, context precision, recall) directly in `eval/metrics/` using the documented formulas, plus the domain metrics. Judge prompts are versioned markdown. Reports are markdown + JSON committed under `eval/reports/<sprint>/`.

Alternatives considered

Option	Why rejected
RAGAS as a dependency	Brings a heavy transitive tree and its own LLM-call conventions for four formulas we can implement in ~200 lines; version churn during a 4-week window; would still not cover the 8 domain metrics
DeepEval / promptfoo	Same coverage gap; another config language to learn

Option	Why rejected
Hosted eval platform (LangSmith datasets, Braintrust)	Data egress of financial queries; external dependency; CI gating becomes network-dependent
Manual spreadsheet scoring	Not repeatable, cannot gate CI

Consequences

Positive: all 16 SLOs in one report, hard gates runnable in CI with no network beyond the LLM provider, metrics are auditable because the formula is in the repo, zero added dependencies beyond numpy.
Negative: we own the metric implementations and their bugs mitigated by unit-testing each metric against hand-computed fixtures; our faithfulness numbers are not strictly comparable to published RAGAS benchmarks, which must be stated in reports.

ADR-013 Hybrid retrieval (dense + sparse + rerank) over pure dense

Status: Accepted **Deciders:** AI Eng lead, DB lead

Context

Financial policy queries mix semantic phrasing with exact tokens: `escalation_flag`, `risk_score`, "Basel III", "liquidity coverage ratio", "BBB rated", specific percentage limits. Pure dense retrieval reliably underperforms on rare exact tokens; Context Precision >=70% and Recall >=75% must hold across both styles.

Decision

Hybrid retrieval in `app/rag/retriever.py`: dense cosine search over `policy_chunks.embedding` (HNSW) unioned with Postgres full-text search (`ts_rank` over a GIN index), fused by weighted score (`HYBRID_KEYWORD_WEIGHT=0.3`, tunable), `FETCH_K=20` candidates reduced to `TOP_K=5` by an LLM cross-encoder-style rerank prompt scoring relevance to the query. Metadata filters by `doc_category` when the Router identifies a clearly scoped intent (e.g. regulatory ? docs 03, 06).

Alternatives considered

Option	Why rejected
Pure dense (top-k cosine)	Misses exact-token queries; measured worse on Basel/threshold probes
Pure BM25	Misses paraphrased policy questions, which are most of C1
Dedicated cross-encoder model	Another model to host; LLM rerank is adequate at 20 candidates
Dense + generous k, no rerank	Precision drops, prompt bloats, latency rises

Consequences

Positive: robust across both query styles; both indexes live in the same database; the weight and k values are tunable knobs during Sprint 5 latency work. **Negative:** rerank adds one LLM call (~200 400ms) acceptable within the 2s budget for C1, and it is the first thing to drop if the budget is threatened (fall back to fused score only); two indexes to maintain on re-ingest.

ADR-014 RBAC enforced at the SQL executor boundary, not in prompts

Status: Accepted **Deciders:** Architect, Security reviewer **Drives:** Unauthorized Data Access = 0

Context

Unauthorized Data Access must be 0 violations "any cross-customer data access is a data breach". An LLM instructed "only show portfolios 1 and 3" will comply almost always, which is not 0.

Decision

`role` and `allowed_portfolio_ids` are resolved by the API auth dependency, carried in `GraphState`, and enforced **inside `app/sqlq/executor.py`**: every `rbac_scoped` template has its portfolio filter injected server-side and every returned row is post-asserted to be in scope (defence in depth). A row out of scope raises, is logged as a security event, and fails the request it is never filtered silently. The RBAC matrix (`analyst` ? assigned portfolios; `risk_manager` ? all portfolios + all risk logs; `compliance` ? all + audit log; `admin` ? admin endpoints, no privileged answer path) lives in `app/core/security.py`. Prompts are never told about scope the model simply cannot obtain out-of-scope rows. Enforcement is implemented from Sprint 4 (state plumbing + executor filter) even though the real auth provider lands in Sprint 8, per risk R11.

Alternatives considered

Option	Why rejected
Prompt-level instruction	Probabilistic; unacceptable for a 0-violation target
Postgres row-level security (RLS) policies	Genuinely stronger, but requires per-request role switching / session GUCs and complicates pooling; **revisit in Sprint 8 as defence in depth**, not as the primary control for now
Filter in the API response layer	Out-of-scope data would already have entered the LLM context that is the breach
Separate database per role	Absurd at this scale

Consequences

Positive: one enforcement point, unit-testable; out-of-scope data never enters an LLM prompt; audit trail of every scoped query; testable by running the whole golden set under three roles and diffing. **Negative:** every new template must declare `rbac_scoped` correctly enforced by a test asserting that every template touching `portfolios`/`portfolio_holdings`/`investment_decisions` is marked scoped; `asset_risk_logs` and `market_indicators` are asset/market-level and not portfolio-scoped, so scoping for them is applied via joins where a portfolio context exists.

ADR-015 — Twelve-factor / cloud-portable runtime

Status: Accepted — **Deciders:** Architect, Backend lead — **See:** `16_Production_and_Cloud_Readiness.md`

Context

The capstone runs on a laptop, but the system must be movable to a cloud VPC without rewriting agents, RAG, or SQL. Hardcoded localhost paths, compose DNS in Python, and writable container disk for sessions are the usual blockers.

Decision

Treat the app as a **stateless 12-factor service** from Sprint 4: - Config only via environment (`pydantic-settings`); `APP_ENV=local|ci|staging|prod` - No local disk required for request handling; sessions in Postgres (ADR-006) - Logs to stdout (JSON); traces via OTLP endpoint URL; metrics on `/metrics` - Backing services addressed by URL (`DATABASE_URL`, `LLM_BASE_URL`, `OTEL_EXPORTER_OTLP_ENDPOINT`) - Same container image for local smoke, CI, staging, and prod — only env and secrets change

Alternatives considered

Option	Why rejected
"Cloud-native rewrite later"	Guarantees a second project; we refuse that debt
Kubernetes-first from day one	Premature; manifests without a chosen provider are theater
Serverless-only (Lambda/Cloud Functions)	LangGraph + connection pooling + multi-second C4 paths fit containers better

Consequences

Positive: cloud move is ops work, not redesign; horizontal scale is replica count. **Negative:** discipline required early (no "quick" absolute paths). Enforced by review checklist in `16`.

ADR-016 — Secrets via env injection; secret manager in cloud; stub auth blocked in prod

Status: Accepted — **Deciders:** Architect, Security reviewer

Context

LLM keys, JWT secrets, and DB passwords must never land in git, images, or traces. Local `.env` is fine for developers; prod needs a secret manager. `AUTH_MODE=stub` is a foot-gun in production.

Decision

- All secrets are env vars listed in `.env.example` (values never committed).
- Cloud: platform injects the same keys from a secret manager (AWS SM / GCP Secret Manager / Azure Key Vault) — **no app code branch per cloud** if injection is env-based.

- `APP_ENV=prod` **refuses to start** if `AUTH_MODE=stub`, if default passwords are detected, or if `JWT_SECRET` is `REPLACE_ME`.
- Structured logs/traces redact keys matching `API_KEY|PASSWORD|SECRET|TOKEN`.

Alternatives considered

Option	Why rejected
Vendor SDK calls inside request path	Couples runtime to one cloud; harder local/CI
Secrets in ConfigMaps / baked image	Common breach pattern
Commit encrypted `.env`	Still a rotation and access-control mess for a student team

Consequences

Positive: one mental model; prod fail-fast prevents stub auth accidents. **Negative:** local onboarding needs a short secrets setup step (documented in `05`).

ADR-017 — One container image; CI gates; deploy manifests separate from app

Status: Accepted — **Deciders:** Architect, Backend lead

Context

Production readiness (T48) and go-live (T56) need a repeatable artifact. Mixing deploy YAML into `app/` and rebuilding differently per environment causes "works on my machine" regressions.

Decision

- **One multi-stage Dockerfile** producing `finance-risk-api:<git-sha>`; non-root user; no secrets in layers.
- **CI (required checks):** unit + escalation/RBAC tests; lint; dependency audit; golden suite on PR or nightly (live LLM optional nightly).
- **`deploy/`** holds compose-prod and optional K8s/ECS stubs — not imported by Python.
- Schema migrate, seed, and ingest are **jobs/CLI**, not side effects of `CMD`.
- Runtime image entrypoint: API only (`uvicorn`).

Alternatives considered

Option	Why rejected
Different images for local vs prod	Drift; evals stop matching
Migrate-on-boot	Dangerous with multiple replicas racing DDL
Full Terraform for three clouds in Sprint 3	YAGNI; resource list in 16 is enough until provider chosen

Consequences

Positive: promote-by-digest; CI is the quality gate; cloud team can take `deploy/` without touching agents. **Negative:** contributors must run migrate/seed explicitly (Makefile targets).

ADR-018 — Corpus behind a storage Protocol (local now, object storage later)

Status: Accepted — **Deciders:** Architect, AI eng

Context

RAG ingest today reads `Documents/`. In cloud, PDFs should live in object storage. If ingest hardcodes `Path("Documents")`, the cloud move forks the pipeline.

Decision

Introduce a thin `CorpusStore` Protocol in `app/core/storage.py`: - `LocalCorpusStore` — filesystem under `data/documents` (default for local/CI) - Optional `S3CompatibleStore` — implemented when cloud cutover starts; unused until then - `rag/ingest.py` depends only on the Protocol

Chunk **embeddings remain in pgvector** (ADR-002); object storage holds source blobs and optional eval artifacts, not the vector index.

Alternatives considered

Option	Why rejected
Always require S3 locally (MinIO)	Extra moving part for every student clone
Embed object URLs only, no local files	Breaks offline/CI without credentials
Put PDF bytes in Postgres BYTEA	Bloats backups; worse for CDN/object lifecycle

Consequences

Positive: cloud corpus swap is one class + env (`CORPUS_BACKEND=local|s3`). **Negative:** tiny abstraction — keep it a Protocol, not a plugin framework.

ADR-019 — Self-serve signup + login; admin assigns scope

Status: Accepted — **Deciders:** Architect, Frontend lead — **Sprint:** 8 (API), 9 (UI)

Context

Analysts need to create accounts without a DBA. Open self-registration that grants `risk_manager` or all portfolios would break Unauthorized Data Access = 0.

Decision

- Public **signup** (`POST /auth/signup`) and **login** (`POST /auth/login`) with email/password.

- New users land as `analyst` with **empty** `portfolio_ids` until an **admin** assigns scope/role.
- Passwords stored as argon2id/bcrypt hashes only; JWT after login (ADR-016).
- Seed one `admin@local` (or env-bootstrapped) account for first login / assignment.
- No social OAuth in capstone; OIDC can replace password login later without changing RBAC.

Alternatives considered

Option	Why rejected
Admin-provisioned users only	Blocks demo UX the product needs ("new user will signup")
Signup with full portfolio access	Violates RBAC / data-isolation SLOs
Magic-link only	Extra email infra for little gain in a capstone

Consequences

Positive: real login/signup screens; safe default scope. **Negative:** empty-scope analysts see limited SQL answers until admin assigns — document in UI.

ADR-020 — Separate auth DB from finance/vector DB

Status: Accepted — **Deciders:** Architect, DB lead, Security — **See:** `14`, `05`, `16`

Context

User credentials, roles, and session turns must not live next to the RAG vector index. Mixing them couples backup/restore of PII with knowledge re-ingest, widens blast radius on SQL mistakes, and blocks scaling auth independently of `pgvector`.

Decision

Two PostgreSQL **databases** (separate connection strings):

Database	Contents	Extension	App access
`finance_auth_db`	`users`, `sessions`, `turns` (and future refresh tokens)	none	read/write via auth pool
`finance_risk_db`	5 finance tables + `policy_chunks` (pgvector)	`vector`	finance: SELECT-only role for query path; migrate role for DDL/seed/ingest

- Env: `AUTH_DATABASE_URL` and `DATABASE_URL` (alias `FINANCE_DATABASE_URL`).
- Local: one Postgres container may host **both databases** (two DBs, not one schema mix). Cloud: may be two managed instances.
- Auth code never opens the vector DB for user rows; RAG/SQL agents never open the auth DB.
- JWT still carries `portfolio_ids` so the finance executor does not need a join to auth mid-query.

Alternatives considered

Option	Why rejected
One DB, `auth` schema + `public` finance	Still one backup/blast domain; user asked for separate DB from vector
Users table inside `finance_risk_db`	Couples PII to vector reindex and read-only finance role complexity
External IdP only (no auth DB)	Still need local user/profile store for portfolio scope in the capstone

Consequences

Positive: clear trust boundary; vector restore without wiping users; auth scalable separately. **Negative:** two pools/migrations; compose must create both DBs. Acceptable.

ADR-021 — Stock screening module (Screener-like UX) with whitelist SQL

Status: Accepted — **See:** `17_Stock_Screening_Feature.md`

Context

Users want Screener.in-style power: define filters on fundamentals and get a stock table (Screener, Create screen). Free-form SQL or scraping Screener would break safety and licensing.

Decision

- Add synthetic `equity_universe` + `equity_fundamentals` in **`finance_risk_db`**.
- Screens are JSON `ScreenDefinition`s compiled by a whitelist SQL builder (ADR-005 family).
- Saved screens in **`finance_auth_db.saved_screens`**.
- Optional NL → definition via Screen agent; UI builder needs no LLM.
- No Screener scraping; educational data only; non-advice disclaimer on every run.

Alternatives considered

Option	Why rejected
Embed Screener iframe / API	Not ours; ToS/licensing; not a capstone system
Text-to-SQL on equity tables	Fails SQL Correctness / injection posture
Vectors for screening	Wrong tool; ratios are structured filters

Consequences

Positive: clear product differentiator; same safety model as portfolio SQL. **Negative:** need seed data + UI; optional golden set beyond mandated 50.

ADR-022 — Capstone and Production are separate plans

Status: Accepted — **See:** `18_Dual_Plan_Capstone_vs_Production.md`, `capstone/`, `production/`

Context

Mixing graded synthetic data with user-owned live portfolios and real-time feeds in one delivery plan confuses submission scope and SLO reproducibility.

Decision

- **Two separate plans:** `planning/capstone/` (NIIT graded) and `planning/production/` (commercial product).
- Capstone does **not** implement workspace import APIs or live providers.
- Production does **not** use the course golden set as its system of record.
- Shared *ideas* (agents, citations, guardrails) may be reused later; planning and prompts stay isolated.
- No `APP_MODE` dual-delivery story for the capstone track.

Alternatives considered

Option	Why rejected
Single app `APP_MODE` switch as the plan	User requested plans kept separate; muddies grading
Only production plan	Fails course dataset/golden-set requirements

Consequences

Positive: clear prompts, clear DoD, no accidental live-data dependency in submission. **Negative:** two docs trees to maintain — acceptable.

ADR-023 — React app: login/signup first; protected routes for all features

Status: Accepted — **See:** `19_React_Frontend_Plan.md`

Context

Users must not access query, screening, or admin features without authentication. A generic "frontend later" plan did not specify React or an auth gate.

Decision

- **React 18 + Vite + TypeScript + React Router** (capstone frontend).
- Public routes: `/`, `/login`, `/signup` only.
- All product features under `/app/*` behind `ProtectedRoute` (JWT required).
- Unauthenticated visits to `/app/*` redirect to `/login`.
- Authenticated users hitting `/login` or `/signup` redirect to `/app`.

Alternatives considered

Option	Why rejected
Streamlit	Poor fit for auth-gated multi-page product UX
Next.js	Extra complexity for capstone; Vite SPA sufficient
Public query page	Violates user requirement; breaks RBAC story

Consequences

Positive: clear security UX; matches signup/login APIs (ADR-019). **Negative:** Sprint 5 must include auth pages even if JWT backend lands Sprint 8 (stub/mock OK until then).

ADR-024 — Cursor agent model: Opus 4.5 or Codex High for planning and implementation

Status: Accepted — **See:** `20_AI_Agent_Model_Policy.md`

Human-driven planning and code generation in Cursor uses **Claude Opus 4.5** (preferred) or **Codex High**. This does not change ADR-004 (runtime LLM for the product).

Superseded / Deferred Decisions

Topic	Status	Note
Postgres RLS as primary RBAC	Deferred to Sprint 8	Considered in ADR-014; may be added as defence in depth
Async job API for C4 queries	Deferred post-Sprint 10	Considered in ADR-010
Long-term / vector memory	Rejected	ADR-006
Model fine-tuning	Out of scope	`01_Project_Approach.md` §2.3
Real-time market data	Capstone: out of scope · Production: separate plan `production/`	ADR-022
Multi-region active-active	Deferred post-cloud	Single region first (16)
Provider-specific Terraform	Deferred until provider chosen	Resource list in 16 only

---

> **Disclaimer.** Synthetic/educational data. Not investment advice.